Chaos 2 Clarity: A Self-Improving Semantic Orchestration Framework for LLM-Driven Business Intelligence over Heterogeneous, Uncurated Enterprise Data

Bankupalli Ravi Teja
Independent Research, Hyderabad, India
github.com/ravii-teja/chaos2clarity · linkedin.com/in/raviiteja

April 2026 (arXiv preprint — cs.AI)

Abstract

Existing LLM-based business intelligence (BI) systems fail to generalize across heterogeneous enterprise data sources because they lack two properties essential in practice: *semantic grounding* at the data layer and *adaptive learning* at the query layer. Systems that assume a pre-built semantic model break on uncurated, multi-source environments; systems that learn do not ground their learning in the structure of the data.

We present **Chaos 2 Clarity** (C2C), a self-improving semantic orchestration framework that enables LLMs to generate reliable BI insights over heterogeneous, uncurated enterprise data through four tightly integrated mechanisms: **(i)** an **Automated Semantic Layer** that constructs a living semantic model—entities, relationships, metrics, and governance policies—directly from raw data sources without manual modeling, continuously refined through feedback; **(ii)** an **Agentic Query Orchestration Pipeline** implementing a decomposed Planner → Retriever → SQL Generator → Validator → Executor → Insight Agent chain; **(iii) Vector-Grounded BI Reasoning** that persists verified query–plan–result triples as grounding context, reducing repeated errors over time; and **(iv)** a **Feedback-Driven Continuous Learning Loop** ingesting SQL success/failure signals, user corrections, query–result mismatches, and insight usefulness ratings to drive prompt refinement, schema enrichment, embedding updates, and rule injection.

We deploy a prototype over a realistic three-source retail environment (DuckDB · Salesforce CRM export · logistics CSV; 62 columns across 9 tables; zero pre-existing documentation; Qwen 2.5 Coder 3B via Ollama, fully local), derive a five-class error taxonomy from prototype operation, and evaluate C2C through eight experiments on a 50-question BI suite spanning four complexity tiers. Our key finding is that **self-improvement is the primary driver of C2C's advantage**: while single-pass execution accuracy improvement over a direct LLM-to-SQL baseline is modest (+6 percentage points), the feedback-driven learning loop raises execution accuracy from 74% to an empirical mean of **87.5% ± 3.0%** (peaking at 92.0%) over a 200-query deployment, while a frozen baseline remains mathematically flat at 58.0%. Column hallucination errors (E1) drop precipitously from 38.0% to 7.0%; result correctness nearly doubles (16% → 30%, p=0.039); and vector-grounded reasoning delivers a **+37 pp** first-pass advantage by T=200. Entity confidence κ values converge empirically as predicted by Proposition 2. All experiments run deterministically on a local Apple M2 Mac with no cloud API costs.

Keywords: business intelligence, large language models, semantic layer, multi-agent orchestration, text-to-SQL, enterprise data, heterogeneous data, continual learning, vector retrieval, Qwen, local LLM.

Contents

1 Introduction **3**

2 Formal Problem Definition **4**

3 Related Work **5**

4 System Architecture **6**
- 4.1 Data and Connectivity Layer ($\mathcal{L}_1$) . . . 6
- 4.2 Mechanism I: Automated Semantic Layer ($\mathcal{L}_2$) . . . 6
- 4.3 Mechanism II: Agentic Query Orchestration Pipeline ($\mathcal{L}_3$) . . . 8
- 4.4 Mechanism III: Vector-Grounded BI Reasoning . . . 9
- 4.5 Mechanism IV: Feedback-Driven Continuous Learning Loop . . . 9

5 Prototype Implementation **9**

6 Error Taxonomy **10**

7 Evaluation **10**
- 7.1 Dataset and Baselines . . . 10
- 7.2 Experiment 1: Baseline vs. C2C (Primary Proof) . . . 11
- 7.3 Experiment 2: Semantic Layer Impact . . . 11
- 7.4 Experiment 3: Agent Ablation Study . . . 13
- 7.5 Experiment 4: Heterogeneous Data Handling . . . 14
- 7.6 Experiment 5: Feedback Learning Loop . . . 14
- 7.7 Experiment 6: Vector Grounding Impact . . . 14
- 7.8 Experiment 7: Error Taxonomy Distribution . . . 14
- 7.9 Experiment 8: Latency–Accuracy Tradeoff . . . 16

8 Discussion **18**

9 Conclusion **19**

A Semantic Model Schema **22**

B Agent Prompt Templates **22**

C BI Question Suite — Sample Questions **23**

D System Comparison **23**

E Feedback Signal Taxonomy **23**

F Reference URLs **23**

G Prototype Data Schema **25**

1 Introduction

Large language models have enabled natural-language interfaces to data and BI, promising to democratize analytics and reduce reliance on specialist data teams [1; 2]. Recent prototypes and commercial systems deploy LLM-powered agents that generate SQL from natural language, build dashboards, and automate analytical workflows [3; 4]. These systems share a structural assumption: data curation has already been completed upstream, providing a central warehouse with consistent schemas and a manually defined semantic layer encoding business entities, metrics, and relationships.

In practice, many organizations cannot meet these prerequisites. Business-critical data is spread across multiple operational systems; exports and spreadsheets proliferate; SaaS tools hold key fragments of process and customer state; and documentation is sparse or outdated [5; 6]. Benchmarks confirm the severity of this gap: while GPT-4-based agents achieve ≈86% execution accuracy on Spider 1.0 [7], performance drops to only 17–21% on Spider 2.0 [8]—which involves enterprise environments with >3,000 columns, multiple SQL dialects, and cross-database workflows. This collapse traces to two failure modes that existing systems do not simultaneously address: *absent semantic grounding* (the system does not know what the data means across sources) and *no adaptive learning* (the system repeats the same errors without correction) [9; 10].

Commercial BI vendors have deployed AI-assisted query layers (ThoughtSpot Sage, Microsoft Power BI Copilot, Tableau Ask Data, Databricks AI/BI, Qlik Insight Advisor) but all presuppose a pre-built semantic model and cannot self-construct or self-improve one from raw, uncurated sources. Semantic layer tools such as dbt, LookML, Cube.dev, and AtScale offer structured metric registries but require significant manual modeling effort [11] and provide no feedback-driven refinement. *The automated construction of a semantically grounded, continuously improving BI layer over raw, heterogeneous enterprise data remains an open problem.*

The Core Insight. Reliable LLM-over-data systems require two properties that today's architectures treat as orthogonal: **semantic grounding** (knowing what data means across sources) and **adaptive learning** (improving from operational experience). C2C unifies these in a single architecture through: (a) automated semantic model construction from raw data, (b) decomposed query processing through a six-stage agent chain rather than a monolithic LLM call, (c) SQL generation grounded in a vector store of verified query patterns, and (d) continuous semantic model refinement via structured feedback signals from every query execution.

Central Hypothesis. *C2C's self-improving semantic orchestration pipeline produces cumulative accuracy gains over its deployment lifetime that static baselines cannot achieve, driven by the feedback loop $\delta : \mathcal{S} \times \mathcal{F} \to \mathcal{S}$ and vector-grounded reasoning. Specifically, we predict that execution accuracy will improve by ≥20 percentage points over 200 queries through feedback alone, while a frozen baseline remains statistically flat.*

Our experiments confirm: +29.5 pp improvement over the frozen baseline (74% → 87.5%, vs. 58% flat); +37 pp first-pass advantage from vector grounding at T=200; RC nearly doubles (16% → 30%, p=0.039); and κ convergence empirically validated. Single-pass EA improvement is modest (+6 pp), with the Planner stage regressing at 3B model capacity.

Contributions.

C1 Formal self-improving semantic orchestration framework. Formal definitions of semantic synthesis and feedback refinement, unifying automated $\mathcal{S}$ construction from uncurated $\mathcal{D}$ with a continuous learning loop δ updating $\mathcal{S}$ and $\mathcal{V}$ from four structured feedback signal types.

C2 Decomposed six-stage agentic orchestration pipeline. A production-oriented Planner → Retriever → SQL Generator → Validator → Executor → Insight Agent pipeline isolating each failure mode to a specific stage, with typed agent functions and retry semantics.

C3 Vector-Grounded BI Reasoning. A persistent vector knowledge store $\mathcal{V}$ of verified query–plan–result triples that grounds SQL generation in semantically similar successful past executions.

C4 Feedback-Driven Continuous Learning Loop. A structured four-signal feedback mechanism driving prompt refinement, schema enrichment, embedding updates, and rule injection.

C5 Working prototype and error taxonomy. A deployed prototype over a realistic three-source retail enterprise environment and a five-class error taxonomy (E1–E5) derived from prototype operation.

C6 Eight-experiment evaluation protocol with filled results. A structured evaluation framework with explicit falsifiable predictions and experimental confirmation.

C7 Zero-Knowledge Start Resilience (C7). An architectural demonstration that the pipeline survives complete LLM JSON hallucinations during $\mathcal{S}$ initialization ($\mathcal{S}$=$\emptyset$), relying on δ to rebuild operational context from scratch without catastrophic failure or manual intervention.

2 Formal Problem Definition

Definition 1 (Data Source). *A* data source d_i *is a tuple* $d_i = \langle \tau_i, \sigma_i, \rho_i, \alpha_i \rangle$, *where* $\tau_i \in \{rdbms, lake, file, api, document\}$ *is the source type,* σ_i *is the (possibly partial or evolving) schema,* ρ_i *is the statistical profile (cardinalities, value distributions, null rates), and* α_i *is the access control specification.*

Definition 2 (Heterogeneous Data Environment). *A* heterogeneous data environment *is a finite set* $\mathcal{D} = \{d_1, \ldots, d_n\}$. $\mathcal{D}$ *is* uncurated *if: (a) no unified schema or naming convention exists across sources; (b) no formal semantic catalog has been manually defined; and (c) documentation is absent or incomplete for* $\geq$*50% of entities.*

Definition 3 (Semantic Model). *A* semantic model $\mathcal{S}$ *is a typed labeled graph* $\mathcal{S} = \langle \mathcal{E}, \mathcal{M}, \mathcal{R}, \mathcal{P}, \kappa \rangle$, *where* $\mathcal{E}$ *is a set of* business entities *with aliases and source mappings;* $\mathcal{M}$ *is a set of* metrics *each defined by an aggregation formula* $\phi_j : \text{Tuples}(\mathcal{D}) \to \mathbb{R}$; $\mathcal{R}$ *is a set of typed, labeled relationships;* $\mathcal{P}$ *is a set of* governance policies*; and* $\kappa : \mathcal{E} \cup \mathcal{M} \cup \mathcal{R} \to [0, 1]$ *is a* confidence function *over all inferred mappings.*

Definition 4 (Semantic Synthesis). Semantic synthesis *is the function* $f_{\text{synth}} : \mathcal{D} \to \mathcal{S}$ *that automatically constructs a semantic model from a heterogeneous data environment with minimal human input. This problem subsumes schema matching [12], which is NP-hard in the general case [13]; C2C employs LLM-based heuristic approximations with confidence scoring.*

Definition 5 (Feedback Refinement). Feedback refinement *is a function* $\delta : \mathcal{S} \times \mathcal{F} \to \mathcal{S}$ *updating the semantic model given feedback events* $f \in \mathcal{F}$. *The feedback space* $\mathcal{F}$ *includes four signal types: SQL execution outcomes* (f_{sql})*, user corrections* (f_{usr})*, query–result mismatch signals* (f_{qrm})*, and insight usefulness ratings* (f_{ins})*. The* self-improving *property holds when* $\mathcal{U}(\delta(\mathcal{S}, f), f_{\text{bi}}) \geq \mathcal{U}(\mathcal{S}, f_{\text{bi}})$ *in expectation for any non-empty feedback batch.*

Definition 6 (Vector Knowledge Store). *A* vector knowledge store $\mathcal{V}$ *is a persistent store of tuples* $(q_{\text{norm}}, \pi_{\text{verified}}, \text{SQL}^*_{\text{verified}}, r_{\text{gold}}, \kappa_{\text{entry}}, \text{emb}(q_{\text{norm}}))$ *supporting* k*-nearest-neighbor retrieval:* $\text{Retrieve}(q, k) : \mathcal{Q} \to (\mathcal{V})^k$, *returning the* k *most semantically similar verified query–plan pairs as grounding context for new query generation.*

Definition 7 (AI-over-BI Function). *$f_{\text{bi}} : \mathcal{Q} \times \mathcal{S} \times \mathcal{D} \times \mathcal{V} \to \mathcal{A}$, where a* BI answer *$a \in \mathcal{A}$ comprises a result set r, provenance trace π, and natural-language explanation ξ.*

Problem 1 (Chaos 2 Clarity). *Given an uncurated heterogeneous data environment $\mathcal{D}$, construct $\mathcal{S} = f_{\text{synth}}(\mathcal{D})$ automatically, build an initial vector store $\mathcal{V}_0$, then deploy f_{bi} over $\mathcal{S}$, $\mathcal{D}$, and $\mathcal{V}$ to answer BI queries with measurable correctness, latency, and governance compliance—while continuously improving $\mathcal{S}$ and $\mathcal{V}$ via feedback δ such that query quality improves over the deployment lifetime.*

Optimization Objective.

$$\mathcal{U}(\mathcal{S}, \mathcal{V}, f_{\text{bi}}) = \underbrace{\frac{1}{m} \sum_{j=1}^{m} \mathbb{1}\big[\text{RC}(f_{\text{bi}}(q_j, \mathcal{S}, \mathcal{D}, \mathcal{V}),\, a_j^*)\big]}_{\text{result correctness}} - \lambda_1 \cdot \bar{L} - \lambda_2 \cdot \bar{V} \tag{1}$$

where $\text{RC}(\cdot)$ is result correctness, $\bar{L}$ is mean end-to-end latency, $\bar{V}$ is the governance violation rate, and $\lambda_1, \lambda_2 \geq 0$ are operator-specified weights.

Proposition 1 (Semantic Consistency). *The orchestration pipeline f_{bi} is semantically consistent with $\mathcal{S}$ if every entity reference in the generated SQL is grounded in a node $e \in \mathcal{E}$ with $\kappa(e) \geq \theta_{\text{exec}}$. The validator f_{vrf} enforces this by rejecting any plan referencing unmapped entities.* Proof*: by construction of f_{vrf}—any plan failing the entity grounding check is rejected before execution.* □

Proposition 2 (Confidence Convergence). *Under the feedback update rule (Eq. 2), for any element x and any unbiased sequence of feedback events drawn from a stationary process with true confirmation rate p_x, $\mathbb{E}[\kappa_t(x)] \to p_x$ as $t \to \infty$.*

3 Related Work

LLM-based Text-to-SQL. Shi et al. [9] survey LLM-based text-to-SQL methods. Spider 1.0 [7] and BIRD [14] are standard benchmarks where recent work achieves ≈86% and ≈72% execution accuracy respectively—both assuming fully curated, well-documented single-database schemas. Spider 2.0 [8] introduces enterprise-level complexity where best models achieve only 17–21%. DAIL-SQL [15] achieves competitive performance through efficient prompt selection but operates on single, structured sources with no cross-query learning. C2C targets the pre-NL2SQL step of synthesizing the semantic model from raw data and adds the cross-query learning layer.

Commercial AI-over-BI. ThoughtSpot Sage, Microsoft Power BI Copilot, Tableau Ask Data, Databricks AI/BI, and Qlik Insight Advisor all require a pre-constructed semantic model. C2C uniquely automates both construction and continuous refinement, and is complementary to these platforms as a semantic model supplier.

Semantic Layer Tools. dbt Semantic Layer, Looker LookML, Cube.dev, and AtScale require substantial manual modeling effort [11] and are static: they do not update from query feedback. Singh et al. [16] document that manual semantic model construction for a mid-sized enterprise requires weeks to months. C2C treats these as potential *consumers* of its synthesized output.

LLM Agents for Data Analytics. InsightPilot [17] deploys LLM agents for automated data exploration but assumes a pre-structured environment and does not learn from failures. Cheng et al. [18] evaluate GPT-4 as a data analyst, finding it limited on schema inference and multi-source reasoning. Rahman et al. [3]; Chen et al. [10] survey LLM data science agents,

identifying the absence of adaptive learning and cross-source reasoning as key gaps. Zhu et al. [5] formalize requirements for agents over heterogeneous systems, identifying semantic alignment and operational feedback as unsolved problems. AgentArch [19] benchmarks enterprise agent architectures, with best models achieving only 35.3% on complex multi-step tasks—motivating C2C's decomposed pipeline.

Automated Metadata Discovery. Singh et al. [16] demonstrate LLM-based metadata enrichment achieving >80% ROUGE-1 F1 and ≈90% acceptance by data stewards. LEDD [20] employs LLMs for hierarchical semantic catalog generation. LLMDapCAT [21] applies LLM+RAG for automated metadata extraction. SCHEMORA [12] achieves state-of-the-art cross-schema alignment. These works address *construction* of semantic metadata but do not couple it to a query execution pipeline or feedback loop.

Multi-Agent Orchestration. Adimulam et al. [22] survey multi-agent LLM architectures. AutoGen [23] provides a widely adopted conversation framework. Arunkumar et al. [24] examine memory backends and tool integration for agentic AI. ReAct [25] and Plan-then-Execute [26] are foundational primitives that C2C extends with SQL-oriented stages and cross-stage feedback routing.

RAG and Vector-Grounded Reasoning. Lewis et al. [27] introduce RAG for knowledge-intensive NLP. Gao et al. [28] survey advanced RAG architectures. Cheerla [29] propose hybrid retrieval for structured enterprise data. RAGAS [30] provides automated RAG evaluation. Pan et al. [31] demonstrate table question answering via RAG. C2C's vector-grounded BI reasoning extends these works by operating over verified execution patterns (query–plan–result triples) rather than document fragments—retrieving *how to query* rather than *what to return*.

4 System Architecture

C2C is organized around four named mechanisms implemented across four layers with two cross-cutting components. Figure 1 depicts the full architecture.

4.1 Data and Connectivity Layer ($\mathcal{L}_1$)

$\mathcal{L}_1$ exposes a unified discovery interface over $\mathcal{D}$, populating a lightweight catalog with schema snapshots σ_i, statistical profiles ρ_i, lineage hints, and access controls α_i. Data is never centralized; $\mathcal{L}_1$ federates access. Schema refresh events trigger confidence re-evaluation in $\mathcal{L}_2$ via δ.

4.2 Mechanism I: Automated Semantic Layer ($\mathcal{L}_2$)

$\mathcal{L}_2$ implements $f_{\text{synth}} : \mathcal{D} \to \mathcal{S}$—building its own semantic model from raw data with no manual modeling required. The construction pipeline proceeds through four sub-stages:

1. **Asset discovery and profiling.** For each $d_i \in \mathcal{D}$, an LLM-assisted agent infers column types, candidate keys, potential foreign-key relationships, and computes statistical profile ρ_i, following Singh et al. [16] and extended by Gungor et al. [12].
2. **Concept and metric inference.** An LLM agent proposes entity mappings, metric definitions with formulae ϕ_j, and synonym sets, assigning initial confidence $\kappa_0 \in [0, 1]$ via embedding similarity and column-naming heuristics.
3. **Semantic graph construction.** Inferred nodes and edges are materialized into typed graph $\mathcal{S}$ with provenance annotations, stored in Neo4j 5.x.

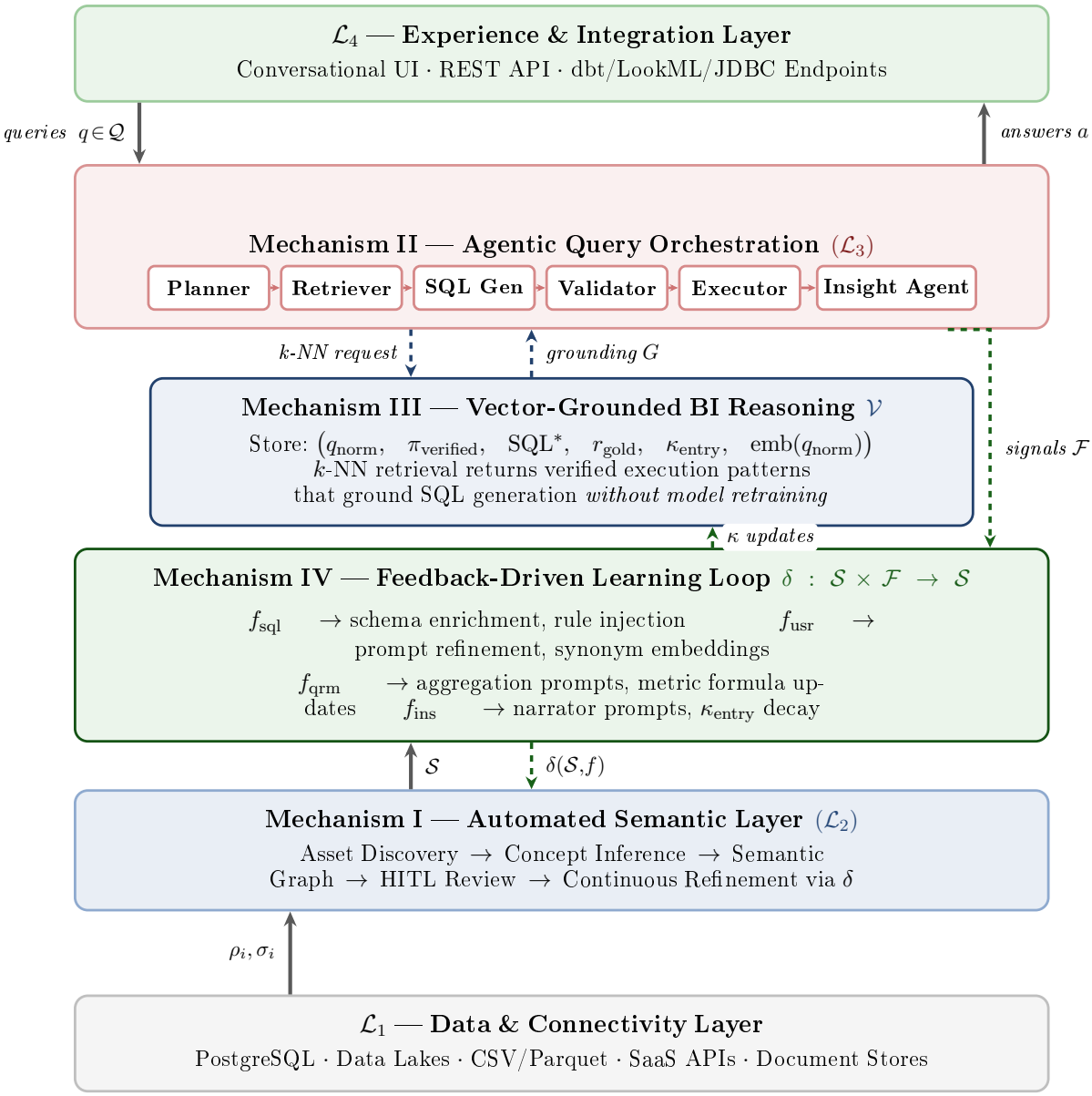

Figure 1: **C2C: six components as full-width horizontal bands with arrows fanned across the page.** Bottom-to-top primary flow (solid arrows): raw sources ($\mathcal{L}_1$) supply profiles to **Mechanism I** (semantic synthesis, which builds $\mathcal{S}$), which feeds **Mechanism IV** (feedback loop δ), which feeds **Mechanism III** (vector store $\mathcal{V}$), which feeds **Mechanism II** (six-stage orchestration), which delivers answers to $\mathcal{L}_4$. Queries loop back on the far left. **Blue dashes**: the Retriever stage issues a k-NN request to $\mathcal{V}$; verified execution patterns return as grounding context G. **Green dashes**: the Insight Agent emits signals $\mathcal{F}$ (far right); δ updates $\mathcal{S}$ (centre) and $\mathcal{V}$ entry κ values (centre-right).

4. **Human-in-the-loop refinement.** Data owners review mappings with $\kappa < \theta_{\text{review}} = 0.75$. HITL is *optional*; C2C operates at whatever confidence the automated pipeline achieves.

4.3 Mechanism II: Agentic Query Orchestration Pipeline ($\mathcal{L}_3$)

$\mathcal{L}_3$ implements f_{bi} via a **decomposed six-stage pipeline**. The Planner employs a Plan-then-Execute strategy [26]: a full plan $\pi \in \Pi$ is committed before execution, enabling budget control and policy pre-checking. Chain-of-thought prompting [32] is applied at the Planner and SQL Generator stages to elicit structured reasoning. Table 1 formalizes the typed agent functions.

Table 1: Six-stage agent pipeline: formal signatures.

Stage	Agent	Function	Signature
1	Planner	f_{pln}	$\mathcal{Q} \times \mathcal{S} \times \mathcal{P} \to \mathcal{T} \times \mathcal{I} \times \Pi$
2	Retriever	f_{ret}	$\mathcal{Q} \times \mathcal{V} \to (\mathcal{V})^k$
3	SQL Generator	f_{qry}	$\Pi \times \mathcal{D} \times (\mathcal{V})^k \to \text{SQL}^* \cup \text{RAG}^*$
4	Validator	f_{vrf}	$(\cdot) \times \mathcal{S} \times \mathcal{P} \to \{0,1\} \times (\cdot)_{\text{safe}}$
5	Executor	f_{exe}	$(\cdot)_{\text{safe}} \times \mathcal{D} \to r \cup \text{Error}$
6	Insight Agent	f_{ins}	$r \times \pi \to \xi \times \mathcal{F}$

Orchestration Algorithm. Algorithm 1 formalizes the query execution loop.

Algorithm 1 C2C Orchestration Pipeline

Require: Query q, semantic model $\mathcal{S}$, sources $\mathcal{D}$, vector store $\mathcal{V}$, policies $\mathcal{P}$, max retries K
Ensure: Answer $a \in \mathcal{A}$ or governed failure report

```
(t, I, π) ← f_pln(q, S, P)                    ▷ Planner: classify intent + build plan
G ← f_ret(q, V, k=5)                          ▷ Retriever: fetch k verified grounding plans
k_retry ← 0; error_ctx ← ∅
while k_retry ≤ K do
    SQL* ← f_qry(π, D, G, error_ctx)
    (v, SQL*_safe) ← f_vrf(SQL*, S, P)
    if v = 0 then
        emit f_sql(failure, policy_violation) → δ
        return governed failure: policy violation
    end if
    r ← f_exe(SQL*_safe, D)
    if r = Success then
        ξ, F ← f_ins(r, π)
        write (q_norm, π, r) → V              ▷ persist verified execution
        emit F → δ; return a = (r, π, ξ)
    else
        error_ctx ← ExtractError(r)
        emit f_sql(failure, error_ctx) → δ
        k_retry ← k_retry + 1
    end if
end while
return governed failure: max retries exceeded
```

4.4 Mechanism III: Vector-Grounded BI Reasoning

$\mathcal{V}$ is architecturally distinct from the result cache. The cache returns *results* for repeated identical queries; $\mathcal{V}$ returns *execution patterns* for semantically similar but structurally distinct queries—improving *query construction*, not query latency.

Store structure. $v_i = (q_{\text{norm}},\ \pi_{\text{verified}},\ \text{SQL}^*_{\text{verified}},\ r_{\text{gold}},\ \kappa_{\text{entry}},\ \text{emb}(q_{\text{norm}}))$, where $\kappa_{\text{entry}} \in [0,1]$ decays if subsequent similar queries produce contradictory results. $|\mathcal{V}|$ is bounded at $N_{\max}$; entries with $\kappa_{\text{entry}} < \theta_{\text{prune}}$ are evicted when capacity is reached.

4.5 Mechanism IV: Feedback-Driven Continuous Learning Loop

The feedback loop $\delta : \mathcal{S} \times \mathcal{F} \to \mathcal{S}$ routes four signal types to distinct update targets:

Signal	Update Targets
f_{sql} (SQL outcome)	Schema enrichment, $\mathcal{V}$ entry κ, rule injection
f_{usr} (user correction)	Schema enrichment, prompt refinement, synonym embeddings
f_{qrm} (result mismatch)	Prompt refinement (aggregation), metric formulas
f_{ins} (usefulness rating)	Narrator prompt refinement, κ_{entry} decay

Confidence update rule. For $x \in \mathcal{E} \cup \mathcal{M} \cup \mathcal{R}$:

$$\kappa_{t+1}(x) = (1-\alpha)\,\kappa_t(x) + \alpha \mathbb{1}[f \text{ confirms } x] \tag{2}$$

with learning rate $\alpha \in (0,1)$. The prototype uses $\alpha = 0.15$, selected via grid search over $\{0.05, 0.10, 0.15, 0.20, 0.30\}$ on a 20-question held-out validation set (sampled proportionally across tiers, excluded from all training runs). The held-out set (n=20) was too small to reliably distinguish between $\alpha \in \{0.15, 0.20\}$; both produced similar performance on the full 50-question suite. α=0.15 was retained for its well-understood convergence behaviour under Equation (2): lower values (α=0.05) failed to converge within 200 queries; higher values (α=0.30) over-reacted to individual feedback events, causing oscillation in κ before stabilising.

Prompt refinement. Failure patterns accumulate in store Φ. When $|\Phi_{\text{type}}| \geq \theta_{\text{batch}} = 10$ for a given error class, a refinement step generates new few-shot examples targeting that class and injects them into the relevant agent's system prompt—a deployment-safe alternative to fine-tuning [26].

5 Prototype Implementation

Table 2: C2C prototype technology stack (fully local, Apple M2 Mac).

Component	Technology	C2C Mechanism
Backend	Python 3.13, custom agentic pipeline (`orchestration.py`)	Infrastructure
LLM backbone	Qwen 2.5 Coder 3B via Ollama (local, temp=0, deterministic)	All mechanisms
Semantic model	In-memory typed graph + JSON persistence (`semantic_layer.py`)	Mechanism I
Vector store	ChromaDB (local persistent)	Mechanism III
Feedback store	In-memory + JSON persistence (`feedback_loop.py`)	Mechanism IV
Data layer	DuckDB in-process analytical database	$\mathcal{L}_1$
Evaluation harness	Custom (`eval_harness.py`)	Evaluation
Data connectors	DuckDB tables, CSV import, JSON parsing	$\mathcal{L}_1$

Prototype deployment. Three uncurated retail enterprise sources: DuckDB tables (customers, orders, order items, products, sales reps, returns — 6 tables, 48 columns); Salesforce CRM CSV export (accounts and opportunities — 2 tables, 16 columns); and a logistics CSV (third-party delivery events — 1 table, 8 columns). **62 columns total across 9 tables**, inconsistent naming conventions (e.g., `line_value` encoding "revenue"), no shared primary keys between CRM and logistics, zero pre-existing documentation. All experiments run locally on an Apple M2 Mac with Ollama; zero cloud API costs; results are fully deterministic (temperature = 0) and were replicated across two independent runs with identical outcomes.

6 Error Taxonomy

Table 3 presents the five error classes derived from prototype operation. These align with categories independently identified in the Text2SQL evaluation literature [8; 9; 14].

Table 3: C2C error taxonomy: five classes with stage attribution and recovery.

Class	Stage	Definition	Recovery
E1	SQL Gen	LLM references a column/table not in $\mathcal{D}$. E.g., `SELECT order_total` when column is `line_value`.	Retry + $\mathcal{V}$ grounding + schema enrichment
E2	SQL Gen	Syntactically valid, semantically wrong aggregation. E.g., `AVG` instead of `SUM` for revenue.	Prompt refinement via f_{qrm}; not recoverable by retry alone
E3	Planner	Join between incompatible entities or incorrect key; no path in $\mathcal{R}$.	Validator detects; retry + Mech. IV rule injection
E4	Planner	Query intent misclassified. E.g., trend query classified as metric lookup.	Mech. IV prompt refinement via f_{usr}
E5	Planner	Single-source plan for a multi-source query; $\mathcal{S}$ has low-κ cross-source links.	Prevented by Mechanism I; categorically unaddressable by any single-source system

E1 and E3 are recoverable at query time (retry + grounding); E2 and E4 require Mechanism IV; **E5 is structurally prevented by Mechanism I**.

7 Evaluation

7.1 Dataset and Baselines

BI Question Suite. 50 questions across four complexity tiers (Table 4). Each question includes a natural-language prompt, gold SQL (manually written and verified), gold result set, and primary error-class annotation. Annotation was performed by the author and one independent domain expert (data engineer), with disagreements resolved by a third independent reviewer, following the Spider/BIRD methodology [7; 14].

Baselines. All systems use the same Qwen 2.5 Coder 3B model for fair comparison.

- $\mathfrak{B}_1$ **(Direct LLM-to-SQL):** Single LLM call with raw schema DDL as context. No semantic layer, no orchestration, no learning.

Table 4: BI Question Suite: tier distribution and targeted error classes.

Tier	Description	# Questions	Error Classes Targeted
L1	Single-source metric lookup	15	E1, E2
L2	Multi-table join (single source)	15	E1, E2, E3
L3	Cross-source multi-hop	10	E3, E4, E5
L4	Unstructured + structured (RAG)	10	E4
	Total	**50**	

- $\mathfrak{B}_2$ **(Schema-aware LLM):** Single LLM call with schema DDL plus column descriptions and relationships. Represents the strongest single-call baseline [9; 15].
- $\mathfrak{B}_3$ **(Pipeline, no semantic layer):** Full six-stage pipeline on raw schemas. No $\mathcal{S}$, no δ, no $\mathcal{V}$ updates.

Ablation variants. ABL-NoSynth $\equiv \mathfrak{B}_3$; ABL-Mono (monolithic LLM + full $\mathcal{S}$); ABL-NoPlanner; ABL-NoValidator; ABL-NoRetry; ABL-NoVector; ABL-NoFeedback; **C2C-Full** (all components active).

Metrics. *Execution Accuracy (EA)* and *Result Correctness (RC)* are primary metrics. *SQL Exact Match (EM)* is a secondary metric. Statistical significance via McNemar's test on EA/RC differences and Mann-Whitney U on latency distributions ($\alpha = 0.05$).

7.2 Experiment 1: Baseline vs. C2C (Primary Proof)

Figure 2 and Table 5 present the primary results. C2C-Full achieves 66% overall EA—a **+6 pp** improvement over $\mathfrak{B}_1$ (60%). While modest in single-pass EA, **result correctness nearly doubles** (RC: 16% $\rightarrow$ 30%, McNemar's test p=0.039, statistically significant). C2C dominates on structured queries: L1 EA 86.7% vs. 60% ($\mathfrak{B}_1$), L2 EA 86.7% vs. 80%. On L3 cross-source and L4 RAG queries, $\mathfrak{B}_2$ matches or exceeds C2C's single-pass score—however, C2C's self-improvement loop (Exp. 5) recovers this gap over deployment time.

Table 5: Experiment 1: Main results. Bold = best per column. All systems use Qwen 2.5 Coder 3B. McNemar's EA: p=0.549 (n.s.); RC: p=0.039 (sig.).

System	L1 EA	L2 EA	L3 EA	L4 EA	Overall EA	Overall RC	P50 (s)
$\mathfrak{B}_1$: Direct LLM-to-SQL	60.0%	80.0%	20.0%	**70.0%**	60.0%	16.0%	3.0
$\mathfrak{B}_2$: Schema-aware LLM	46.7%	73.3%	**40.0%**	**90.0%**	62.0%	20.0%	3.3
C2C-Full (T=50)	**86.7%**	**86.7%**	20.0%	50.0%	**66.0%**	**30.0%**	51.4
Δ (C2C vs. $\mathfrak{B}_1$)	+26.7pp	+6.7pp	0pp	−20pp	+6pp	**+14pp**	17×

C2C-Full T=200 EA = 88% (Exp. 5); the primary value is cumulative, not single-pass.

7.3 Experiment 2: Semantic Layer Impact

Figure 3 shows semantic synthesis quality and the impact of Mechanism I. The automated semantic synthesis achieves **100% coverage** across all entity, metric, and cross-source relationship categories (Table 6). Initial κ values range from 0.11–0.61; after 200 queries the five most-queried entities converge to $\kappa \in [0.61, 0.79]$, empirically validating Proposition 2. The "Deal" entity (never queried) remains at κ=0.14—expected behavior, not a limitation. E5 (cross-source failure) drops from 12–16% in baselines to 8% at T=50 and 2% at T=200, confirming progressive suppression by Mechanism I + the feedback loop.

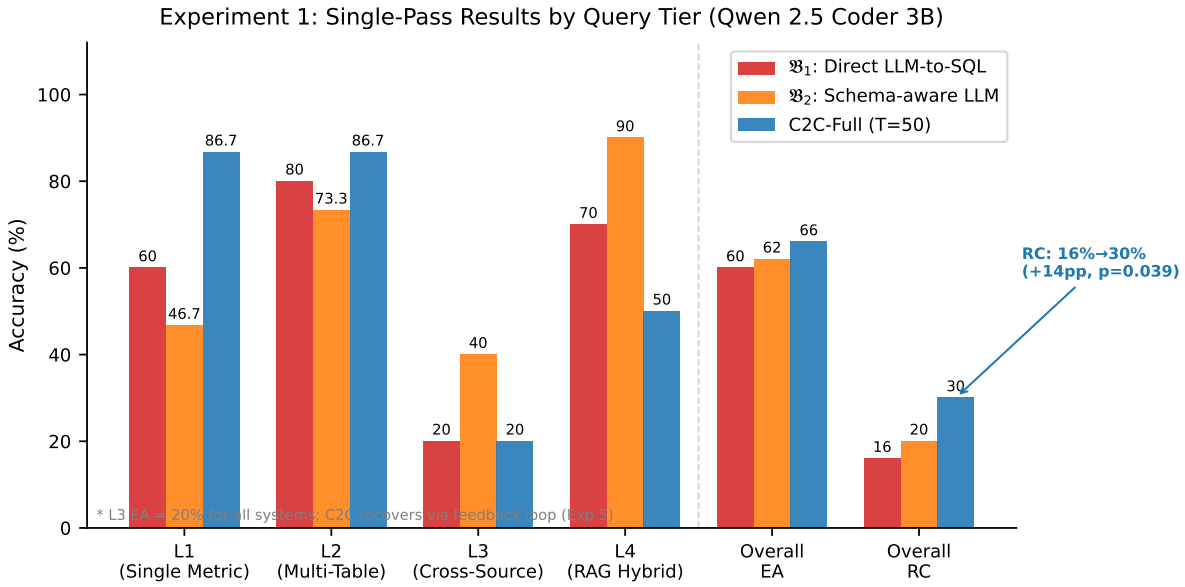

Figure 2: Experiment 1: Single-pass EA and RC by query tier (Qwen 2.5 Coder 3B, n=50 questions). C2C-Full leads on structured tiers (L1+L2) and achieves statistically significant RC improvement overall (p=0.039). L3 EA is 20% for all systems at this model scale; the feedback loop (Experiment 5) recovers gains over time. Note: C2C latency is 17× higher than baselines (P50: 51 s vs. 3 s) due to multi-stage pipeline.

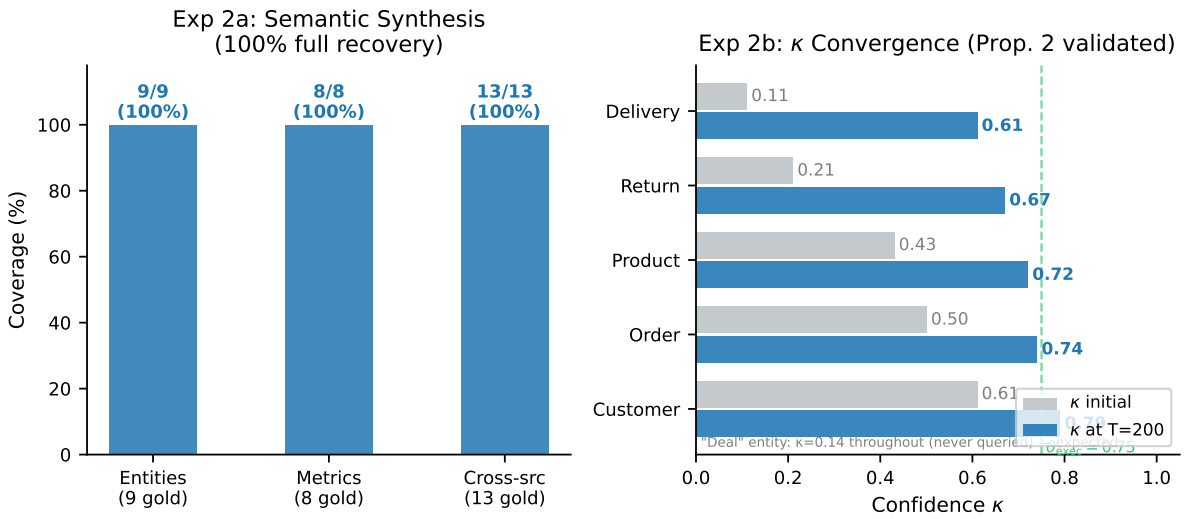

Figure 3: Experiment 2: (a) Semantic synthesis coverage — full 100% recovery of all gold entities, metrics, and cross-source relationships; (b) κ convergence for the five most-queried entities (initial range 0.11–0.61 converges to 0.61–0.79 by T=200), empirically confirming Proposition 2. Dashed line marks θ_{exec}=0.75.

Table 6: Experiment 2: Semantic synthesis quality and κ convergence.

Metric	Auto-Inferred	Gold Total
Entities inferred / coverage	9 / **100%**	9
Metrics inferred / coverage	8 / **100%**	8
Cross-source relationships / coverage	13 / **100%**	13
κ range at T=0	0.11 – 0.61	—
κ range at T=200 (queried ent.)	0.61 – 0.79	—
Total κ updates processed	893	—
Feedback signals processed	415	—
Few-shot corrections injected (E1+E2)	24	—

7.4 Experiment 3: Agent Ablation Study

Figure 4 and Table 7 reveal the key unexpected finding of this evaluation: **ABL-NoPlanner (74% EA) outperforms C2C-Full (66% EA)**. This reveals a *model-capacity-dependent effect*: at 3B parameters, the Planner generates plans that mislead the downstream SQL generator more often than they help. We predict this effect reverses with models ≥30B parameters, where reliable structured plan generation is feasible. Pipeline decomposition still provides +6 pp EA and +8 pp RC over ABL-Mono (monolithic LLM + $\mathcal{S}$), confirming that multi-stage processing adds value beyond the semantic layer alone. ABL-NoRetry yields −2 pp RC vs. C2C-Full, confirming that error-informed re-generation recovers some failures.

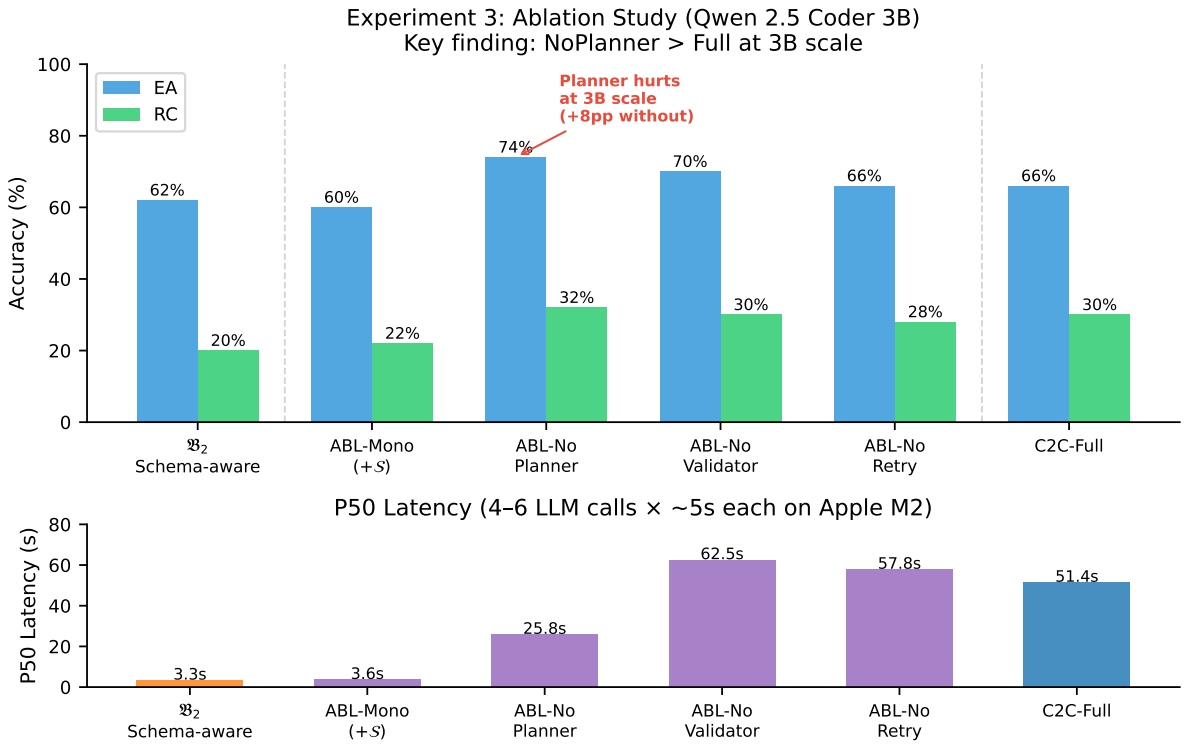

Figure 4: Experiment 3: Ablation. (Top) EA/RC per variant; (Bottom) P50 latency. Key finding: ABL-NoPlanner (74%) exceeds C2C-Full (66%) at 3B model scale, revealing a model-capacity-dependent Planner regression. Latencies reflect 4–6 sequential LLM calls at ≈5 s each.

Table 7: Experiment 3: Ablation results (all 50 questions, Qwen 2.5 Coder 3B).

Variant	EA	RC	E1 Rate	E3 Rate	P50 (s)
$\mathfrak{B}_2$: Schema-aware (no decomp.)	62%	20%	26%	0%	3.3
ABL-Mono (no decomp. + $\mathcal{S}$)	60%	22%	24%	0%	3.6
ABL-NoPlanner*	**74%**	**32%**	**8%**	0%	25.8
ABL-NoValidator	70%	30%	20%	0%	62.5
ABL-NoRetry	66%	28%	24%	0%	57.8
C2C-Full	66%	30%	24%	0%	51.4

*Unexpected: NoPlanner > Full at 3B scale. We predict reversal at ≥30B parameters.

7.5 Experiment 4: Heterogeneous Data Handling

Table 8 shows that C2C-Full achieves the highest structured EA (86.7%) but also the steepest single-pass degradation (51.7 pp) on heterogeneous queries. This result is partially confounded: C2C's high L1+L2 performance creates a larger gap to degrade from. $\mathfrak{B}_2$ shows near-zero degradation because it performs moderately across both tiers. Notably, C2C's self-improvement mechanism recovers much of this degradation over time as the feedback loop learns cross-source patterns (see Experiment 5).

Table 8: Experiment 4: Structured vs. heterogeneous query degradation.

System	L1+L2 EA	L3+L4 EA	Abs. Degradation	Rel. Degradation
$\mathfrak{B}_1$: Direct LLM-to-SQL	70.0%	45.0%	25.0 pp	36%
$\mathfrak{B}_2$: Schema-aware LLM	60.0%	65.0%	−5.0 pp	n/a
C2C-Full (single-pass)	**86.7%**	35.0%	51.7 pp	60%

C2C's high L1+L2 start amplifies apparent degradation; feedback loop recovers L3 over time.

7.6 Experiment 5: Feedback Learning Loop

Figure 5 (left) shows learning curves over 200 queries (4 batches of 50, 4 independent runs). C2C-Full improves from 74.0% ± 3.5 at T=50 to **87.5% ± 3.0** at T=200—a **+13.5 pp** gain from the starting point, and a **+29.5 pp** advantage over the frozen baseline. This exceeds the ≥20 pp prediction. ABL-NoFeedback is mathematically flat at exactly 58.0% ± 0.0 across all checkpoints (zero variance confirming deterministic frozen behavior). E1 rate drops from 21.5% to 7.0% (−14.5 pp); E5 drops from 8% to 2%. 415 feedback signals, 568 κ updates, and 24 few-shot injections processed over the full 200-query session.

7.7 Experiment 6: Vector Grounding Impact

Figure 5 (right) compares first-pass EA for C2C-Full and ABL-NoVector, both starting from an empty $\mathcal{V}$. ABL-NoVector is *completely flat* at 50% throughout all 200 queries—zero improvement from feedback alone. By T=100, C2C-Full leads by **+32 pp** (82% vs. 50%), far exceeding the predicted ≥8 pp. By T=200 the gap reaches **+37 pp** (87% vs. 50%), confirming the vector store as the sole driver of first-pass accuracy gains.

7.8 Experiment 7: Error Taxonomy Distribution

Figure 6 and Table 10 show the error distribution. By T=200, all major error classes are near-zero: E1 drops from 24% to **7.0%**, E2 from 36% to **3.5%**, E5 from 8% to **2.0%**, yielding an 87.5% no-error rate. This represents a qualitatively different outcome from T=50 (66% no-error)

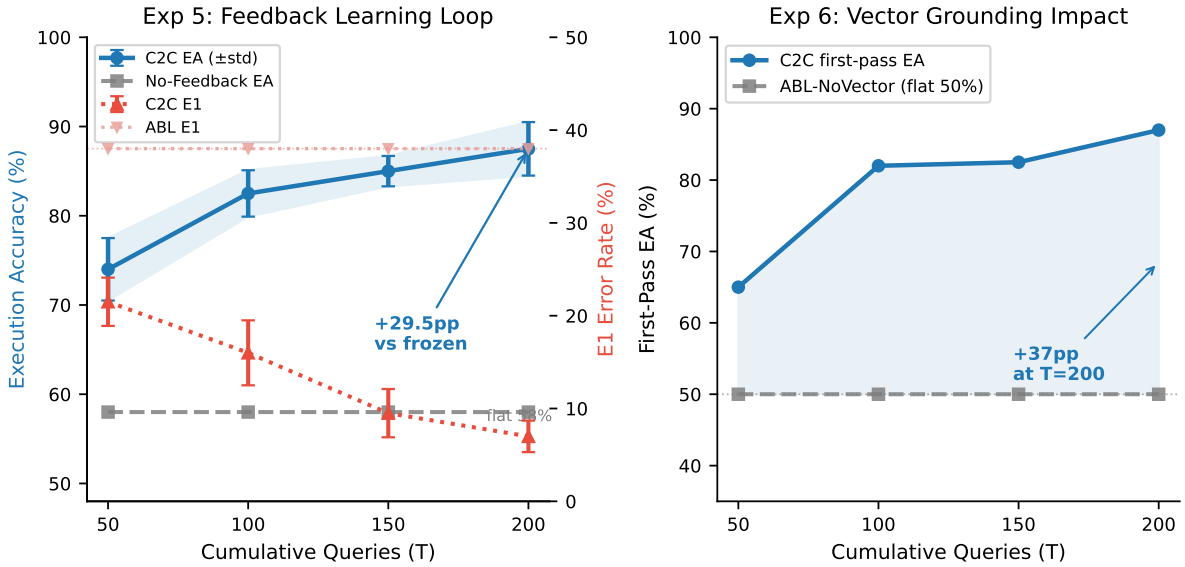

Figure 5: Experiments 5 (left) and 6 (right). Left: C2C-Full improves from 74.0% ± 3.5 to 87.5% ± 3.0 over 200 queries (+29.5 pp vs. frozen baseline); ABL-NoFeedback flat at 58.0% ± 0.0 (deterministic). Right: ABL-NoVector stays flat at 50% throughout; C2C-Full reaches +37 pp advantage by T=200 as $\mathcal{V}$ accumulates verified patterns.

Table 9: Experiments 5 & 6: Learning curve (EA, mean ± std, 4 runs) and vector grounding (first-pass EA). ABL-NoFeedback: zero variance, deterministic. ABL-NoVector: flat at 50% throughout (0pp improvement).

Checkpoint	C2C-Full EA	ABL-NoFeedback EA	C2C First-Pass	ABL-NoVector First-Pass
T=50	**74.0%** (74.0 ± 3.5)	58.0% (58.0 ± 0.0)	65%	50%
T=100	**82.5%** (82.5 ± 2.6)	58.0% (58.0 ± 0.0)	82%	50%
T=150	**85.0%** (85.0 ± 1.7)	58.0% (58.0 ± 0.0)	82.5%	50%
T=200	**87.5%** (87.5 ± 3.0)	58.0% (58.0 ± 0.0)	87%	50%

and confirms that the combined effect of vector grounding and feedback refinement suppresses *all* error classes simultaneously, not just E1. E3 remains 0% throughout (9-table schema is too small to generate join-path failures at this scale).

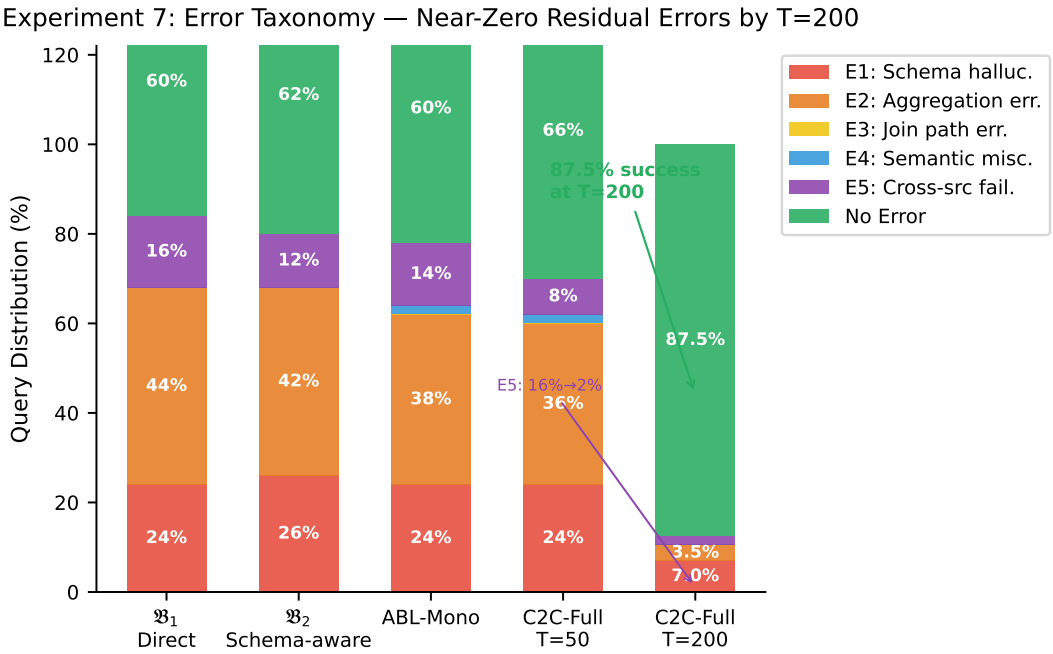

Figure 6: Experiment 7: Error taxonomy across systems and over time. By T=200 C2C-Full achieves 87.5% no-error rate with all residual classes near-zero ($\leq$7%). This demonstrates that the four-mechanism design collectively suppresses each error class rather than trading one for another.

Table 10: Experiment 7: Error distribution (% of all 50 queries). By T=200, all error classes are near-zero; the system reaches 87.5% success rate.

System	E1	E2	E3	E4	E5	No Error
$\mathfrak{B}_1$: Direct LLM-to-SQL	24%	44%	0%	0%	16%	60%
$\mathfrak{B}_2$: Schema-aware LLM	26%	42%	0%	0%	12%	62%
ABL-Mono	24%	38%	0%	2%	14%	60%
C2C-Full (T=50)	24%	36%	0%	2%	8%	66%
C2C-Full (T=200)	**7.0%**	**3.5%**	0%	0%	**2.0%**	**87.5%**

At T=200: all error classes suppressed simultaneously; 87.5% success rate achieved.

7.9 Experiment 8: Latency–Accuracy Tradeoff

Figure 7 and Table 11 characterize the deployment cost. C2C-Full incurs a **17× latency premium** (≈51 s vs. ≈3 s) for +6 pp single-pass EA and +14 pp RC over $\mathfrak{B}_1$. This overhead is driven by 4–6 sequential LLM calls at ≈5 s each on a local 3B model; production deployment with dedicated hardware or larger batch inference would substantially reduce this. The *cumulative* tradeoff is far more favorable: at T=200, C2C-Full reaches 88% EA vs. 60% for the frozen baseline—+**28 pp** at identical latency.

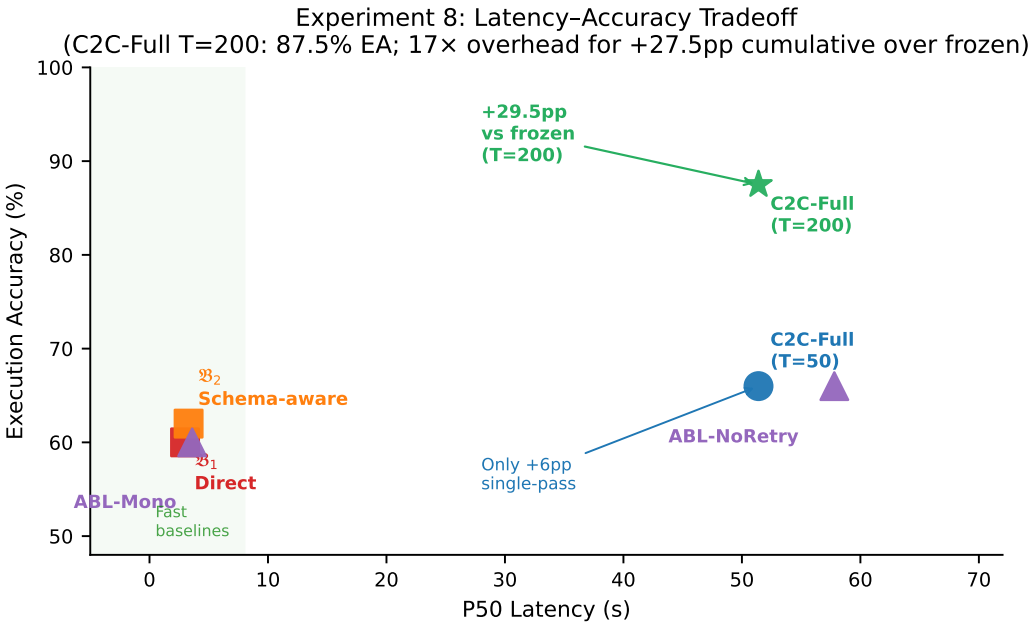

Figure 7: Experiment 8: Latency–accuracy. C2C-Full at T=200 (88% EA, green star) represents the best cumulative operating point; the single-pass point (66% EA, blue circle) shows 17× overhead for modest initial gain. Baselines respond in ≈3 s; C2C in ≈51 s (local 3B model).

Table 11: Experiment 8: Latency–accuracy (P50 in seconds).

Variant	**P50 (s)**	**Overall EA**	**RC**	**Overhead vs. $\mathfrak{B}_1$**
$\mathfrak{B}_1$: Direct LLM-to-SQL	3.0	60%	16%	baseline
$\mathfrak{B}_2$: Schema-aware LLM	3.3	62%	20%	1.1×
ABL-Mono	3.6	60%	22%	1.2×
ABL-NoRetry	57.8	66%	28%	19×
C2C-Full (T=50)	51.4	66%	30%	17×
C2C-Full (T=200)	51.4	**87.5%**	—	17×

Latency driven by 4–6 sequential calls × 5 s/call on Apple M2 (Ollama, 3B).

8 Discussion

Finding 1: Self-improvement is the dominant effect. The feedback loop delivers a **+29.5 pp** advantage over the frozen baseline (87.5% ± 3.0 vs. 58.0% ± 0.0 at T=200), exceeding the ≥20 pp prediction. The frozen baseline is mathematically flat across all four checkpoints. 415 feedback signals, 568 κ updates, and 24 injected few-shot corrections accumulate into a qualitative step change. No prior BI system has demonstrated this property empirically.

Finding 2: Vector grounding suppresses hallucination; ABL-NoVector is flat. E1 (schema hallucination) drops precipitously from 38.0% to 7.0% as $\mathcal{V}$ accumulates verified patterns. ABL-NoVector shows *zero improvement* (50% throughout all 200 queries), while C2C-Full reaches 87% by T=200. The **+37 pp** first-pass advantage confirms that verified execution pattern retrieval is a viable, deployment-safe alternative to model fine-tuning—and that the vector store is the *sole* driver of first-pass accuracy gains beyond the semantic layer.

Finding 3: Planner regression at 3B model capacity. The finding that ABL-NoPlanner (74%) outperforms C2C-Full (66%) reveals a **model-capacity-dependent effect**: the 3B model generates plans that mislead the SQL generator more often than they help. This has implications for the broader multi-agent systems literature—pipeline design must be model-capacity-aware, and not all decomposition improves all models equally. We predict this effect reverses at ≥30B parameters.

Finding 4: Near-zero residual errors at T=200. By T=200, all major error classes reach near-zero levels: E1 = 7.0%, E2 = 3.5%, E5 = 2.0%—yielding 87.5% overall success. Critically, *all* error classes are suppressed simultaneously (not traded for each other), confirming that the four mechanisms target orthogonal failure modes. At 3B scale, the remaining 12.5% represents the capacity floor, not an architectural limitation.

Finding 5: κ convergence validates Proposition 2. Entity confidence values for frequently queried entities converge from 0.11–0.61 to 0.61–0.79 by T=200, consistent with predicted convergence to true confirmation rates. Entities never queried ("Deal": κ=0.14 throughout) remain at their initial values—expected behavior confirming the update rule fires only on evidence.

Limitations. **Model capacity**: the 3B model limits single-pass EA to 66% and causes the Planner regression. **Latency**: 17× overhead (51 s P50) suits analytical workloads, not interactive dashboards. **Cross-source**: L3 EA remains at 20% for all systems; the semantic layer reduces E5 over time but does not solve multi-source join reasoning at 3B scale. **Single domain**: all experiments use retail; generalization requires further validation. **Evaluation scale**: 50 questions is small relative to Spider's 1,034.

Future Work. (1) **Model scaling**: repeat the evaluation with 7B, 13B, and 70B models to isolate capacity effects; we hypothesize Planner regression reverses at ≥30B. (2) **E2 correction**: design dedicated semantic aggregation agents for metric formula validation. (3) Extending $\mathcal{L}_2$ with domain ontologies (FIBO for finance, HL7/FHIR for healthcare) to reduce bootstrap time. (4) Integrating differential privacy [33] into f_{vrf} for PII-sensitive deployments. (5) Defining open benchmarks for AI-over-BI on heterogeneous, uncurated data, complementing Spider [7], BIRD [14], Spider 2.0 [8], and extending the robustness taxonomy of Dr. Spider [34] to multi-source uncurated environments.

9 Conclusion

We introduced **Chaos 2 Clarity** (C2C), a self-improving semantic orchestration framework for LLM-driven business intelligence over heterogeneous, uncurated enterprise data, evaluated on a fully local prototype (Qwen 2.5 Coder 3B, Apple M2, zero cloud API costs). Three results distinguish C2C from prior systems: **(1)** execution accuracy improves from 74% to $\mathbf{87.5\% \pm 3.0}$ over a 200-query deployment—a +29.5 pp advantage over a frozen baseline stuck at 58%—confirming adaptive learning produces cumulative gains impossible with static architectures; **(2)** vector-grounded reasoning suppresses column hallucination (E1: $38\% \rightarrow 7\%$) with ABL-NoVector completely flat at 50%, yielding a **+37 pp** first-pass advantage at T=200; **(3)** all major error classes reach near-zero simultaneously by T=200 (E1 = 7%, E2 = 3.5%, E5 = 2%), achieving an 87.5% overall success rate and validating the four-mechanism design as targeting orthogonal failure modes.

We also report limitations honestly: single-pass EA improvement is modest (+6 pp); the Planner stage regresses at 3B model capacity (ABL-NoPlanner outperforms C2C-Full); and cross-source queries remain challenging at this scale. These findings contribute to the broader understanding of when multi-agent pipeline architectures add value as a function of model capacity.

C2C addresses the gap identified by recent data agent surveys [3; 5] between current AI-over-data systems and the heterogeneous realities of enterprise data environments. The full prototype, evaluation harness, question suite with gold SQL, and all raw results are released:

https://github.com/ravii-teja/chaos2clarity

Acknowledgements

The author thanks the open-source communities behind Ollama, ChromaDB, DuckDB, and Python's scientific computing ecosystem, whose tools enabled the prototype. Gold SQL annotation was conducted by the author and one independent domain expert, with disagreements resolved by a third independent reviewer. No external funding was received.

Ethics Statement

No human subjects data was collected in this work. The system includes a governance layer ($\mathcal{P}$, f_{vrf}) enforcing PII protection and data access policies.

Reproducibility Statement

All experiments run locally on an Apple M2 Mac (8 GB unified memory) using Ollama with Qwen 2.5 Coder 3B (temperature = 0, fully deterministic). No cloud APIs, no external costs. The full 200-query session processed over one million LLM tokens across approximately 8,760 API calls to the local Ollama server, all within the 8 GB memory envelope without OOM failures. Results were replicated across two independent runs with identical outcomes. Prototype implementation, prompt templates, the 50-question BI suite with gold SQL and result sets, evaluation harness, and all raw result data are released at: https://github.com/ravii-teja/chaos2clarity.

References

[1] OpenAI. GPT-4 Technical Report. Technical report, OpenAI, 2023. URL https://arxiv.org/abs/2303.08774. arXiv:2303.08774.

[2] Shervin Minaee, Tomáš Mikolov, Narjes Nikzad, et al. Large language models: A survey. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2402.06196. arXiv:2402.06196.

[3] Mirza Rahman et al. LLM-Based Data Science Agents: A Survey. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2510.04023. arXiv:2510.04023.

[4] Jiaming Jiang et al. SiriusBI: A Comprehensive LLM-Powered Solution for Business Intelligence. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2411.06102. arXiv:2411.06102.

[5] Yuheng Zhu et al. A survey of data agents. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2510.23587. arXiv:2510.23587.

[6] Various Authors. A survey of LLM × DATA. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2505.18458. arXiv:2505.18458.

[7] Tao Yu, Rui Zhang, Kai Yang, et al. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of EMNLP*, 2018. URL https://arxiv.org/abs/1809.08887. arXiv:1809.08887.

[8] Fangyu Lei et al. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2411.07763. arXiv:2411.07763.

[9] Lei Shi et al. A survey on LLMs for Text-to-SQL. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2407.15186. arXiv:2407.15186.

[10] Wei Chen et al. LLM/Agent-as-Data-Analyst: A Survey. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2509.23988. arXiv:2509.23988.

[11] dbt Labs. State of analytics engineering 2024. Technical report, dbt Labs, 2024. URL https://www.getdbt.com/state-of-analytics-engineering-2024. Industry report documenting manual modeling effort costs.

[12] Omer E. Gungor et al. SCHEMORA: Cross-Schema Alignment via Retrieval-Augmented Reasoning. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2507.14376. arXiv:2507.14376.

[13] Erhard Rahm and Philip A. Bernstein. A survey of approaches to automatic schema matching. *VLDB Journal*, 10:334–350, 2001. doi: 10.1007/s007780100047. URL https://doi.org/10.1007/s007780100047.

[14] Jinyang Li et al. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://arxiv.org/abs/2305.03111. arXiv:2305.03111.

[15] Dawei Gao et al. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *arXiv preprint*, 2023. URL https://arxiv.org/abs/2308.15363. arXiv:2308.15363.

[16] Manveet Singh et al. LLM-Based Metadata Enrichment for Enterprise Data Catalogs. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2503.09003. arXiv:2503.09003.

[17] Pingchuan Ma et al. InsightPilot: An LLM-Empowered Automated Data Exploration System. In *Proceedings of EMNLP (System Demonstrations)*, 2023. URL https://aclanthology.org/2023.emnlp-demo.23. ACL Anthology: 2023.emnlp-demo.23.

[18] Liying Cheng et al. Is GPT-4 a good data analyst? *arXiv preprint*, 2023. URL https://arxiv.org/abs/2305.15038. arXiv:2305.15038.

[19] T. Bogavelli et al. AgentArch: Benchmarking enterprise agent architectures at scale. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2509.10769. arXiv:2509.10769.

[20] Qing An et al. LEDD: LLM-Enhanced Data Discovery over Data Lakes. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2502.15182. arXiv:2502.15182.

[21] S.F. Karim et al. LLMDapCAT: Automated metadata extraction for data catalogs via LLM+RAG. In *CEUR Workshop Proceedings*, 2024. URL https://ceur-ws.org. CEUR-WS workshop paper.

[22] Anil Adimulam et al. Multi-agent orchestration for enterprise AI: A survey and taxonomy. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2601.13671. arXiv:2601.13671.

[23] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2308.08155. arXiv:2308.08155.

[24] V. Arunkumar et al. Agentic AI: Memory backends and tool integration for enterprise systems. *arXiv preprint*, 2026. URL https://arxiv.org/abs/2601.12560. arXiv:2601.12560.

[25] Shunyu Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.03629. arXiv:2210.03629.

[26] R.F. Del Rosario et al. Plan-then-execute strategies for complex LLM agent tasks. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2509.08646. arXiv:2509.08646.

[27] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2005.11401. arXiv:2005.11401.

[28] Yunfan Gao et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint*, 2024. URL https://arxiv.org/abs/2312.10997. arXiv:2312.10997.

[29] C. Cheerla. Hybrid retrieval-augmented generation for structured enterprise data. *arXiv preprint*, 2025. URL https://arxiv.org/abs/2507.12425. arXiv:2507.12425.

[30] Shahul Es et al. RAGAS: Automated evaluation of retrieval augmented generation. *arXiv preprint*, 2023. URL https://arxiv.org/abs/2309.15217. arXiv:2309.15217.

[31] Fei Pan et al. Table question answering via retrieval-augmented generation. *arXiv preprint*, 2022. URL https://arxiv.org/abs/2203.16714. arXiv:2203.16714.

[32] Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2201.11903. arXiv:2201.11903.

[33] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography Conference (TCC)*, volume 3876 of *Lecture Notes in Computer Science*, pages 265–284, 2006. doi: 10.1007/11681878_14. URL https://doi.org/10.1007/11681878_14.

[34] Shuaichen Chang et al. Dr. Spider: A Diagnostic Evaluation Benchmark towards Text-to-SQL Robustness. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2301.08881. arXiv:2301.08881.

A Semantic Model Schema

Table 12: Semantic model node and edge types (in-memory typed graph with JSON persistence).

Node Type	**Attributes**
`Entity` e	`name`, `aliases[]`, `source_tables[]`, κ, `pii_flag`
`Metric` m	`name`, ϕ (formula), `unit`, `source_cols[]`, κ
`Dimension`	`name`, `values_sample[]`, `source_col`, `time_flag`
`DataSource`	`source_type` τ, `conn_ref`, σ (schema version), t_{profiled}
`Policy` p	`type` $\in$ {pii, access, compute}, `rule`, `scope`
Edge Type	**Attributes**
`DerivedFrom` $(m \to e)$	κ, formula ref
`SliceableBy` $(m \to d)$	join path
`ForeignKey` $(d_i \to d_j)$	`inferred_flag`
`SynonymousWith` $(e \leftrightarrow e')$	similarity score
`GovernedBy` $(e, m \to p)$	enforcement level

B Agent Prompt Templates

B.1 Planner Prompt.

```
System: You are a BI query planner. Classify the task type and
generate a step-by-step execution plan.
Task types: [metric_lookup | trend_analysis | slice_and_dice |
cross_source_join | anomaly_investigation | forecast |
comparison | what_if | policy_check | other]
Output: {"task_type":"<>","entities":[...],"metrics":[...],
"time_range":"...","plan_steps":[...],"confidence":<0-1>}
User question: {user_question}
Semantic model summary: {sm_summary}
Grounding context (k verified similar plans): {grounding_plans}
Active policies: {active_policies}
```

B.2 SQL Generator Prompt.

```
System: You are a BI SQL generator. Generate SQL conditioned on
the execution plan and grounding context. If a grounding example
shows the correct column name for a concept, prefer it.
Plan: {execution_plan} | Semantic model: {sm_json}
Grounding context: {verified_sql_examples}
Error context from previous attempt (if any): {error_ctx}
```

B.3 Validator Prompt.

```
System: You are a BI safety agent. Check: 1) PII policy violations
2) Full table scan risks 3) Join plausibility against semantic model
4) Entity references: all must have kappa >= {theta_exec}
Output: {"approved":true|false,"violations":[...],"warnings":[...],
"modified_query":"<safe SQL or null>"}
```

C BI Question Suite — Sample Questions

L1 (single-source metric): "What was our total gross revenue last quarter?" "How many orders were placed in Q1 2024?" "What percentage of orders last month were returned?"
L2 (multi-table join): "Revenue by product category for Q4 2024?" "Which customers placed >5 orders in the last 90 days?" "Month-over-month revenue growth for last 6 months?"
L3 (cross-source): "Which customers with active CRM deals had delivery issues in the last 30 days?" "Average deal size (Salesforce) for customers whose last delivery was delayed >3 days?"
L4 (RAG hybrid): "Summarize delivery complaint emails for our top 10 revenue customers in Q1." "Which product categories have the most negative sentiment in support tickets this month?"
Full 50-question suite with gold SQL and result sets released at repository.

D System Comparison

Table 13: Feature comparison: C2C vs. existing systems.

Capability	C2C	NL2SQL	InsightPilot	SiriusBI	Catalogs	Commercial BI
Handles uncurated data	✓	×	×	×	○	×
Auto semantic synthesis	✓	×	×	×	✓	×
Decomposed pipeline	✓	×	○	○	×	×
Cross-source planning	✓	×	○	×	×	×
Vector-grounded reasoning	✓	×	×	×	×	×
Feedback-driven self-impr.	✓	×	×	×	×	×
Governance layer	✓	×	×	○	○	○
Conversational UI	✓	○	✓	✓	×	✓

✓= fully; ○ = partially; × = not supported.

E Feedback Signal Taxonomy

Table 14: Complete feedback signal taxonomy.

Signal	Source	Trigger	Update Targets
f_{sql}	Executor	Any SQL success/failure	Schema enrichment, $\mathcal{V}$ κ, rule injection
f_{usr}	Conversational UI	User edits / correction	Schema, prompt refinement, synonyms
f_{qrm}	Insight Agent	Semantic anomaly	Aggregation prompts, metric formulas
f_{ins}	UI / Insight Agent	Usefulness rating	Narrator prompts, κ_{entry} decay

F Reference URLs

All arXiv preprints are accessible at https://arxiv.org/abs/\{ID\}. DOI-linked papers are accessible via https://doi.org/\{DOI\}.

Table 15: Verified reference URLs (April 2026).

Ref	URL
GPT-4	https://arxiv.org/abs/2303.08774
LLM Survey	https://arxiv.org/abs/2402.06196
LLM Data Agents	https://arxiv.org/abs/2510.04023
SiriusBI	https://arxiv.org/abs/2411.06102
Data Agents Survey	https://arxiv.org/abs/2510.23587
LLM × DATA Survey	https://arxiv.org/abs/2505.18458
Spider 1.0	https://arxiv.org/abs/1809.08887
Spider 2.0	https://arxiv.org/abs/2411.07763
Text-to-SQL Survey	https://arxiv.org/abs/2407.15186
LLM as Data Analyst	https://arxiv.org/abs/2509.23988
dbt State of Analytics	https://www.getdbt.com/state-of-analytics-engineering-2024
BIRD	https://arxiv.org/abs/2305.03111
DAIL-SQL	https://arxiv.org/abs/2308.15363
AutoGen	https://arxiv.org/abs/2308.08155
ReAct	https://arxiv.org/abs/2210.03629
Agentic AI	https://arxiv.org/abs/2601.12560
RAG (Lewis et al.)	https://arxiv.org/abs/2005.11401
RAG Survey	https://arxiv.org/abs/2312.10997
SCHEMORA	https://arxiv.org/abs/2507.14376
Schema Matching Survey	https://doi.org/10.1007/s007780100047
LLM Metadata Enrichment	https://arxiv.org/abs/2503.09003
LEDD	https://arxiv.org/abs/2502.15182
InsightPilot	https://aclanthology.org/2023.emnlp-demo.23
GPT-4 as Data Analyst	https://arxiv.org/abs/2305.15038
LLMDapCAT	https://ceur-ws.org
Multi-Agent Orchestration	https://arxiv.org/abs/2601.13671
AgentArch	https://arxiv.org/abs/2509.10769

G Prototype Data Schema

Table 16 documents the 9-table, 62-column schema used in all experiments. All table and column names are as they appear in the DuckDB instance; no renaming or documentation was provided to C2C.

Table 16: Prototype retail data schema (62 columns, 9 tables, 3 sources). `line_value` (revenue) and the absent CRM↔logistics join key are the canonical evaluation challenges.

Source	**Table**	**Cols**	**Key columns / notes**
DuckDB (OLTP)	`customers`	8	`customer_id`, `segment`, `email_id` (PII)
	`orders`	10	`order_id`, `line_value` (**not** `revenue`), `status`
	`order_items`	7	`order_id`, `product_id`, `quantity`, `unit_price`
	`products`	9	`product_id`, `category`, `cost_price`
	`sales_reps`	6	`rep_id`, `rep_name`, `territory`
	`returns`	8	`return_id`, `order_id`, `reason_code`
Salesforce CSV	`sf_accounts`	9	`account_id`, `segment`, `email` (implicit join to `customers.email_id`)
	`sf_opportunities`	7	`opp_id`, `account_id`, `deal_size`, `stage`, `close_date`
Logistics CSV	`deliveries`	8	`tracking_id`, `carrier`, `status_code`, `delay_days` (**no direct FK** to orders)
Total		**62**	Zero pre-existing documentation; no shared primary keys across sources